End-to-End Secure IoT Firmware Compiler

Compiler Design — Execution Plan & Final Report

Submitted by	B. Amardeep
Roll Number	24CSB0B13
Class	CSE-B
Submitted to	Prof. Preeti Soni
Department	Computer Science & Engineering

Date	January 2026

Abstract

IoT firmware routinely ships with hardcoded secrets, weak cryptography, and insecure communication protocols, making billions of devices vulnerable. This project builds a security-aware compiler for C/C++ IoT firmware that enforces end-to-end security at *compile time* — detecting vulnerabilities before the firmware is ever deployed. The compiler integrates a 9-phase pipeline covering lexical secret detection, AST-based semantic analysis, taint-tracking data-flow analysis, a configurable policy enforcement engine, and energy-aware code transformations. Evaluation on representative Contiki-NG, Zephyr, and ESP-IDF style firmware shows 100% assertion accuracy on an 8-case test suite with only ∼7–12% compilation overhead versus a real GCC baseline.

Contents

1 Problem Definition & Context (Week 1) 4

1.1 Problem Statement . . . 4
1.2 Threat Landscape . . . 5
1.3 Current Challenges . . . 5
1.4 Proposed Solution . . . 5
1.5 Scope . . . 6

2 Literature Survey & Gap Analysis (Week 2) 7

2.1 Research Gap Table . . . 8
2.2 Existing Static Analysis Tools . . . 9
2.3 Existing Secure Compilers . . . 9
2.4 Research Gap . . . 9
2.5 Research Contribution . . . 10

3 Requirements & System Design (Weeks 3–4) 11

3.1 Functional Requirements . . . 11
3.1.1 FR1: Lexical Security Analysis . . . 11
3.1.2 FR2: Semantic Security Analysis . . . 11
3.1.3 FR3: Data-Flow Analysis . . . 11
3.1.4 FR4: Policy Enforcement . . . 11
3.1.5 FR5: Energy-Aware Optimisation . . . 12
3.2 Non-Functional Requirements . . . 12
3.3 Security Policy Grammar (Week 4) . . . 12

4 Front-End Implementation (Weeks 5–6) 14

4.1 Security-Aware Lexer (Week 5) . . . 14
4.1.1 Token Classes . . . 14
4.1.2 Secret Detection Patterns . . . 14
4.1.3 Insecure Function Detection . . . 15
4.1.4 Weak RNG Detection . . . 15
4.2 Parser and AST Generation (Week 6) . . . 15
4.2.1 AST Node Structure . . . 15

4.2.2 Security Annotations . . . 16
4.2.3 Sample AST Output . . . 16

5 Security Analysis & Enforcement (Weeks 7–9) 17
5.1 Symbol Table with Security Attributes (Week 7) . . . 17
5.1.1 Symbol Structure . . . 17
5.1.2 Scope Management . . . 17
5.2 Data-Flow Analysis — Taint Tracking (Week 8) . . . 18
5.2.1 Three-Phase Process . . . 18
5.2.2 Safe Source Whitelist . . . 18
5.2.3 Example: Detected Secret Leak . . . 18
5.3 Semantic Analysis (Week 9 — Policy Enforcement) . . . 19
5.3.1 Checks Performed . . . 19
5.3.2 Authentication Policy Check . . . 19

6 Policy-Driven Compilation & Optimisation (Week 10) 20
6.1 Energy-Aware Security Transformations . . . 20
6.1.1 Transformations Applied . . . 20
6.1.2 Energy Profile Comparison . . . 21

7 Testing & Validation (Week 11) 22
7.1 Test Suite Design . . . 22
7.2 Test Cases . . . 22
7.3 Coverage Summary . . . 22
7.4 Violations Detected in Provided Test Files . . . 23

8 Performance & Overhead Evaluation (Week 12) 24
8.1 Methodology . . . 24
8.2 Compilation Time Results . . . 24
8.3 Phase-Level Breakdown . . . 25
8.4 Memory Overhead . . . 25

9 System Architecture & Design (Complete View) 27
9.1 Nine-Phase Compilation Pipeline . . . 27
9.2 Module Responsibilities . . . 28

10 Conclusion & Future Work (Weeks 13–14) 29
10.1 Summary of Achievements . . . 29
10.2 Key Results . . . 29
10.3 Limitations . . . 30
10.4 Future Work . . . 30

References **32**

A Project File Structure **33**

B Quick Start **35**

Chapter 1

Problem Definition & Context (Week 1)

1.1 Problem Statement

The Internet of Things ecosystem is growing exponentially, with billions of embedded devices deployed across smart homes, industrial systems, healthcare, and critical infrastructure. Despite this growth, IoT firmware security remains critically underdeveloped. Devices routinely ship with:

- **Hardcoded credentials** — API keys, passwords, and tokens embedded directly in source code
- **Weak cryptographic implementations** — use of broken algorithms such as DES, MD5, and RC4
- **Insecure communication** — MQTT without TLS, CoAP without DTLS, plain HTTP
- **Buffer overflow vulnerabilities** — use of dangerous C functions such as `gets()`, `strcpy()`, `sprintf()`
- **Missing authentication** — connections made without credentials or certificates

Traditional compilers (GCC, Clang) focus exclusively on *correctness* and *optimisation*, completely ignoring these security concerns. By the time a firmware image is deployed, vulnerabilities are expensive or impossible to patch.

1.2 Threat Landscape

Table 1.1: IoT Firmware Threat Categories and Mitigations

Threat	Impact	Compiler Mitigation
Hardcoded credentials	Full device takeover, lateral movement	Static secret detection at lexical phase
Weak cryptography (DES, MD5)	Data decryption, forgery	Crypto strength checks in semantic phase
No TLS/DTLS	Eavesdropping, MITM attacks	Protocol enforcement in policy engine
Buffer overflow	Remote code execution (RCE)	Insecure function replacement in transformer
Missing authentication	Unauthorised device access	Authentication policy enforcement
Secret data leaks via printf	Credential exposure in logs	Taint-tracking data-flow analysis

1.3 Current Challenges

1. **Compile-time security gap** — no existing general-purpose compiler performs security-aware analysis
2. **Resource constraints** — IoT devices have severely limited memory, CPU, and battery; security overhead must be minimal
3. **Supply chain risk** — insecure firmware propagates through update and build pipelines without detection
4. **Protocol diversity** — MQTT, CoAP, HTTP each have distinct security requirements that no single tool handles together

1.4 Proposed Solution

A security-aware compiler that integrates security enforcement directly into the compilation pipeline, providing:

- Vulnerability detection at compile time — not post-deployment
- Automatic security policy enforcement (configurable)
- Energy-aware transformations for resource-constrained devices
- Secure firmware output as default, not as an afterthought

1.5 Scope

- **Language:** C/C++ subset commonly used in IoT firmware
- **Target platforms:** ESP32, ARM Cortex-M, Contiki-NG, Zephyr RTOS (software-only, no hardware dependency)
- **Security scope:** Compile-time static enforcement only (dynamic analysis is out of scope)

Chapter 2

Literature Survey & Gap Analysis (Week 2)

2.1 Research Gap Table

Table 2.1: Research Gap Analysis: Existing Literature

Paper / Tool	Contribution	Limitation
Leroy (2009) CompCert	Formally verified C compiler guaranteeing semantic preservation	No security rules; no IoT focus
Wurm et al. (2016)	Security analysis of consumer and industrial IoT devices	Post-deployment only; no compile-time enforcement
Checkoway et al. (2011)	Comprehensive automotive firmware attack surface study	Domain-specific; no general-purpose secure compiler
Alrawi et al. (2019)	Systematic evaluation of home IoT security across firmware and network	Survey only; no compiler-level mitigation
Clang Static Analyzer	Open-source static analysis for C/C++; detects memory bugs	No IoT, crypto, or protocol rules; no energy awareness
Coverity (Synopsys)	Industry-grade static analysis with CWE mapping	Commercial; no IoT protocol awareness or transformation
OWASP IoT Top 10 (2023)	Defines top 10 IoT risks including hardcoded credentials	Guidelines only; no automated enforcement tool
Schwartz et al. (2010)	Foundational taint analysis and symbolic execution study	Runtime-based; overhead too high for IoT devices

2.2 Existing Static Analysis Tools

Table 2.2: Comparison of Existing Static Analysis Tools

Tool	Strengths	Weaknesses
Clang Static Analyzer	Good general bug detection, open source	No IoT-specific rules; no protocol awareness
Coverity	Comprehensive, industry-proven	Expensive; not IoT-focused; no energy awareness
SonarQube	Good UI; continuous integration support	Requires separate runtime; no compile-time enforcement
Flawfinder	Fast, lightweight C/C++ analysis	Only flags functions, no semantic context
PC-lint	Thorough type checking	Commercial; no cryptographic or protocol rules

2.3 Existing Secure Compilers

Table 2.3: Comparison of Existing Secure Compilation Approaches

Compiler / Approach	Security Features	IoT Relevance
CompCert	Formally verified compilation	No security rules; no IoT focus
SMAC Compiler	Side-channel protection	Research prototype; not production-ready
GCC `-fstack-protector`	Stack smashing protection	Limited scope; no crypto or protocol checks
Clang `-fsanitize`	Runtime sanitisers	Runtime only; adds overhead unacceptable for IoT
SPARK/Ada	Strong typing, formal proofs	Not C/C++; steep learning curve for IoT devs

2.4 Research Gap

No existing tool provides:

1. Combined secret detection + protocol security + energy optimisation in a single

compilation pass
2. Policy-driven compilation configurable per deployment scenario
3. Compile-time TLS/DTLS enforcement for IoT-specific protocols (MQTT, CoAP)
4. Energy-security trade-off optimisation (e.g. selecting ChaCha20 over AES-256 for battery-operated devices)

2.5 Research Contribution

This project bridges the gap between compiler technology and IoT security:

- First end-to-end secure compiler targeting IoT C/C++ firmware
- Novel integration of taint analysis + policy enforcement in a lightweight pipeline
- Protocol-aware security rules (MQTT, CoAP, HTTP)
- Energy-aware security transformations based on device power profile
- Open-source prototype suitable for both research and education

Chapter 3

Requirements & System Design (Weeks 3–4)

3.1 Functional Requirements

3.1.1 FR1: Lexical Security Analysis

- Detect hardcoded secrets: API keys, passwords, JWT tokens, private keys, AWS credentials
- Flag insecure C functions: `gets()`, `strcpy()`, `strcat()`, `sprintf()`, `scanf()`
- Identify weak random number generators: `rand()`, `random()`, `srand()`

3.1.2 FR2: Semantic Security Analysis

- Check cryptographic algorithm strength — reject DES, 3DES, RC4, MD5, SHA-1
- Validate protocol security — flag `mqtt://`, `coap://`, `http://`
- Detect missing authentication in MQTT and CoAP connections
- Check for missing error handling in TLS/crypto operations

3.1.3 FR3: Data-Flow Analysis

- Track secret propagation across variable assignments (taint analysis)
- Detect sensitive data leaks via `printf`, `fprintf`, `mqtt_publish`, `coap_send`
- Mark secret sources and propagate taint through the symbol table

3.1.4 FR4: Policy Enforcement

- TLS mandatory for MQTT (`mqtt://` → `mqtts://`)
- DTLS mandatory for CoAP (`coap://` → `coaps://`)

- No hardcoded credentials or cryptographic keys
- Strong cryptography only (AES-256, ChaCha20, SHA-256)
- Authentication required for all broker connections

3.1.5 FR5: Energy-Aware Optimisation

- Profile-based crypto algorithm selection
- Adaptive logging level configuration
- Protocol security upgrade transformations
- Insecure function replacement

3.2 Non-Functional Requirements

Table 3.1: Non-Functional Requirements with Targets

Requirement	Description	Target
Compilation time	Security analysis per 500 lines	< 50 ms
Memory overhead	Peak process memory	< 50 MB
Detection accuracy	True positive rate	$> 90\%$
False positive rate	Incorrect flags on secure code	$< 10\%$
GCC overhead	Extra time vs. plain `gcc -O0`	$< 20\%$

3.3 Security Policy Grammar (Week 4)

The compiler uses a configurable policy dictionary. The following EBNF defines the policy language:

```
PolicyFile      = { PolicyRule }
PolicyRule      = RuleName ":" Condition [ "->" Action ]
RuleName        = identifier
Condition       = "HARDCODED_SECRET"  | "WEAK_RANDOM"
                | "NO_TLS"            | "WEAK_CRYPTO"
                | "NO_AUTH"           | "INSECURE_FUNCTION"
Action          = "ERROR" | "WARN" | "FIX" | "IGNORE"
```

Listing 3.1: Security Policy Grammar (EBNF)

In practice these policies are expressed as a Python dictionary in `config.py`:

```
SECURITY_POLICIES = {
    "NO_HARDCODED_KEYS":         True,   # Block hardcoded crypto keys
    "TLS_REQUIRED":              True,   # MQTT/HTTP must use TLS
    "NO_WEAK_RANDOM":            True,   # Block rand(), random()
    "NO_INSECURE_FUNCTIONS":     True,   # Block gets(), strcpy() etc.
    "NO_HARDCODED_CREDENTIALS": True,   # Block passwords in source
    "DTLS_REQUIRED_FOR_COAP":    True,   # CoAP must use DTLS
    "STRONG_CRYPTO_ONLY":        True,   # Block DES, MD5, RC4 etc.
    "MUTUAL_AUTH_REQUIRED":      False,  # Optional: mTLS enforcement
}
```

Listing 3.2: Active Security Policy Configuration

Chapter 4

Front-End Implementation (Weeks 5–6)

4.1 Security-Aware Lexer (Week 5)

The lexer performs two jobs simultaneously: standard C tokenisation and security-rule detection. It processes the source line by line using compiled regular expressions.

4.1.1 Token Classes

Table 4.1: Token Classes Recognised by the Security-Aware Lexer

Token Type	Example
KEYWORD	int, const, void
IDENTIFIER	API_KEY, mqtt_connect
STRING	"sk_live_4eC39Hq..."
NUMBER	42, 0xFF
PREPROC	#include, #define
OPERATOR	=, ==, !=
PUNCTUATION	(,), ;
COMMENT	// ..., /* ... */

4.1.2 Secret Detection Patterns

```
SECRET_PATTERNS = [
    (r'(api[_-]?key)\s*=\s*["\']([A-Za-z0-9+/=]{20,})["\']',
     "Hardcoded API key"),
    (r'(password|passwd)\s*=\s*["\']([^"\']{4,})["\']',
     "Hardcoded password"),
    (r'(token|jwt)\s*=\s*["\']([A-Za-z0-9\-_.]{20,})["\']',
     "Hardcoded token/JWT"),
    (r'=\s*"-----BEGIN (RSA|EC) PRIVATE KEY-----',
```

```
     "Embedded private key"),
    (r'aws[_-]?access[_-]?key[_-]?id\s*=\s*["\']([A-Z0-9]{16,})["\']',
     "AWS Access Key"),
]
```

Listing 4.1: Secret Detection Regex Patterns

4.1.3 Insecure Function Detection

Using word-boundary regex to avoid substring false positives:

```
INSECURE_FUNCTIONS = [
    "gets", "strcpy", "strcat", "sprintf",
    "scanf", "sscanf", "memcpy"
]
# Matched with: re.search(rf'\b{func}\s*\(', line)
```

Listing 4.2: Insecure Function Detection

4.1.4 Weak RNG Detection

```
SECURE_RNG = ['esp_random', 'getrandom', 'RAND_bytes', 'arc4random']

def detect_weak_random(self):
    for line_num, line in enumerate(self.lines, 1):
        if any(s in line for s in SECURE_RNG):
            continue   # skip secure lines
        for rng in WEAK_RANDOM_FUNCTIONS:
            if re.search(rf'\b{re.escape(rng)}\s*\(', line):
                # report issue ...
```

Listing 4.3: Weak RNG Detection with Secure Whitelist

4.2 Parser and AST Generation (Week 6)

The parser converts the token stream into a security-annotated Abstract Syntax Tree. Security-relevant nodes receive `security_attrs` metadata.

4.2.1 AST Node Structure

```python
class ASTNode:
    def __init__(self, node_type, value=None, line=None):
        self.type            = node_type     # PROGRAM, DECLARATION, etc.
        self.value           = value         # token value
        self.line            = line          # source line number
        self.children        = []            # child nodes
        self.symbols         = {}            # linked symbol entries
        self.security_attrs  = {}            # e.g. {"requires_secure": True}
```

Listing 4.4: AST Node Class

4.2.2 Security Annotations

Critical IoT API calls receive automatic security annotations during parsing:

```python
SECURITY_SENSITIVE_CALLS = [
    "mqtt_connect", "coap_send", "tls_handshake",
    "encrypt", "decrypt"
]

def parse_function_call(self):
    name_token = self.consume('IDENTIFIER')
    node = ASTNode("FUNCTION_CALL", name_token.value, name_token.line)
    if name_token.value in SECURITY_SENSITIVE_CALLS:
        node.security_attrs["requires_secure"] = True
    # parse arguments ...
    return node
```

Listing 4.5: Automatic Security Annotation in Parser

4.2.3 Sample AST Output

For the code `const char* API_KEY = "sk_live_xyz";`, the AST produced is:

```
PROGRAM
  DECLARATION: const
    VARIABLE: API_KEY    [line 1]
    INIT_VALUE: "sk_live_xyz"
```

Listing 4.6: Sample AST (tree representation)

Chapter 5

Security Analysis & Enforcement (Weeks 7–9)

5.1 Symbol Table with Security Attributes (Week 7)

The symbol table extends the classical compiler symbol table with security-specific fields.

5.1.1 Symbol Structure

```
class Symbol:
    def __init__(self, name, sym_type, scope, line):
        self.name              = name
        self.type              = sym_type
        self.scope             = scope
        self.line              = line
        self.initialized       = False
        self.initial_value     = None
        self.is_secret         = False    # taint source flag
        self.is_constant       = False
        self.data_flow_source = None     # tracks taint origin
```

Listing 5.1: Symbol Class with Security Fields

5.1.2 Scope Management

The symbol table uses a scope stack. Variables are marked as secret during the data-flow analysis phase and this marking propagates to all tainted copies.

```
class SymbolTable:
    def enter_scope(self): ...           # push new scope
    def exit_scope(self):  ...           # pop scope
    def add(self, name, sym_type, line): ...
    def lookup(self, name): ...          # walks scope stack
    def mark_as_secret(self, name): ... # taint marking
```

```
    def get_all_secrets(self): ...        # returns tainted symbols
```

Listing 5.2: Symbol Table Scope and Taint API

5.2 Data-Flow Analysis — Taint Tracking (Week 8)

The data-flow analyser tracks how secret values propagate through the program and detects when they reach an output sink.

5.2.1 Three-Phase Process

1. **Source marking** — variables assigned high-entropy string literals are tagged as tainted (excluding safe URL prefixes such as `mqtts://`)
2. **Taint propagation** — assignments of the form `dest = tainted_source` propagate the taint to `dest`
3. **Sink detection** — if any tainted variable appears in an output call (`printf`, `fprintf`, `mqtt_publish`), a DATA_LEAK issue is raised

5.2.2 Safe Source Whitelist

```
SAFE_VALUE_PREFIXES = ['mqtts://', 'coaps://', 'https://']
SAFE_RETRIEVAL_FNS  = [
    'secure_storage_read', 'nvs_read_string',
    'secure_get_', 'tls_get_'
]
# Lines containing these are skipped during source marking
```

Listing 5.3: False Positive Prevention in Taint Source Detection

5.2.3 Example: Detected Secret Leak

```
const char* API_KEY = "sk_live_4eC39HqLyjWDarjtT1zdp7dc";

void connect_mqtt() {
    // VIOLATION: secret leaked via printf
    printf("Connecting with API key: %s\n", API_KEY);
}
```

Listing 5.4: Input firmware with secret leak (test.c)

Issue reported:

```
[CRITICAL] DATA_LEAK: Secret 'API_KEY' leaked via printf()  (line
    5)
```

5.3 Semantic Analysis (Week 9 — Policy Enforcement)

The semantic analyser and policy engine together check higher-level security properties that cannot be detected purely at the lexical level.

5.3.1 Checks Performed

Table 5.1: Semantic and Policy Checks

Check	Severity	Trigger Condition
Insecure protocol (MQTT)	CRITICAL	`mqtt://` URL present
Insecure protocol (CoAP)	CRITICAL	`coap://` URL present
Insecure protocol (HTTP)	HIGH	`http://` URL present
Weak cipher	CRITICAL	DES, 3DES, RC4 referenced
Weak hash	HIGH	MD5, SHA-1 called
Missing authentication	HIGH	`mqtt_connect()` with no credentials
Missing error handling	MEDIUM	Crypto call with no `if` check after
Buffer overflow risk	HIGH	Fixed array + unsafe input on next line

5.3.2 Authentication Policy Check

```python
def check_missing_authentication(self):
    for line_num, line in enumerate(self.lines, 1):
        # Exact match for bare mqtt_connect (not
    mqtt_connect_auth)
        if re.search(r'\bmqtt_connect\s*\(', line):
            has_username = 'username' in line.lower()
            has_cert     = 'cert' in line.lower()
            if not (has_username or has_cert):
                self.issues.append({
                    "type": "POLICY_VIOLATION",
                    "policy": "AUTHENTICATION_REQUIRED",
                    "severity": "HIGH", ...
                })
```

Listing 5.5: MQTT Authentication Check in Policy Engine

Chapter 6

Policy-Driven Compilation & Optimisation (Week 10)

6.1 Energy-Aware Security Transformations

The transformer applies both security fixes and energy-profile-driven optimisations to produce the output firmware.

6.1.1 Transformations Applied

1. **Insecure function replacement**
   `gets(buf)` → `fgets(buf, sizeof(buf), stdin)`
   `strcpy(d,s)` → `strncpy(d,s,sizeof(d)-1)`
   `sprintf(...)` → `snprintf(..., sizeof(buf), ...)`

2. **Weak RNG replacement**
   `rand()` → `SECURE_RANDOM()` (macro that selects `esp_random()` on ESP32, `getrandom()` on Linux)

3. **Protocol URL upgrade**
   `mqtt://` → `mqtts://`
   `coap://` → `coaps://`
   `http://` → `https://`

4. **Energy-aware crypto selection** (based on power profile)
   AES-256-CBC → ChaCha20 for `ultra_low_power` and `battery_operated` profiles

5. **Logging level configuration**
   Injects a compile-time `#define LOG_LEVEL` appropriate to the selected energy profile

6. **Error handling injection**
   Wraps bare crypto calls with return-value checks

6.1.2 Energy Profile Comparison

Table 6.1: Energy Profile Settings and Use Cases

Profile	Crypto	Logging	Use Case
`ultra_low_power`	ChaCha20	ERROR only	Coin-cell sensors
`battery_operated`	ChaCha20	WARNING+	Portable IoT devices
`mains_powered`	AES-256	INFO+	Gateways and hubs

Why ChaCha20 for low-power devices?
ChaCha20 is a software-friendly stream cipher that performs approximately 40% fewer operations per byte than AES-256-CBC on devices without hardware AES acceleration (most Cortex-M0/M0+ class MCUs). This directly translates to lower energy per encryption operation and extended battery life.

Chapter 7

Testing & Validation (Week 11)

7.1 Test Suite Design

The test suite contains 8 assertion-based test cases drawn from real IoT firmware patterns used in Contiki-NG, Zephyr RTOS, ESP-IDF, and bare-metal embedded projects. Each test case specifies an *expected verdict* (PASS or FAIL); a test *passes* only when the compiler's actual verdict matches the expectation.

7.2 Test Cases

Table 7.1: Week 11 Test Suite Results

#	Firmware	Platform	Expected	Actual	Issues
1	Temp sensor (insecure)	Contiki-NG	FAIL	FAIL	10
2	Light node (secure)	Contiki-NG	PASS	PASS	0
3	MQTT gateway (weak crypto)	Zephyr	FAIL	FAIL	8
4	CoAP sensor (no DTLS)	Zephyr	FAIL	FAIL	7
5	OTA update (cred leak)	ESP-IDF	FAIL	FAIL	5
6	Secure provisioning	ESP-IDF	PASS	PASS	0
7	Buffer overflow risk	Bare-metal	FAIL	FAIL	7
8	Reference secure FW	Bare-metal	PASS	PASS	0
	Total			**8/8**	**37**

Result: 8/8 assertions passed (100% assertion accuracy).

7.3 Coverage Summary

- ✓ Hardcoded secret / credential detection
- ✓ Insecure function detection (`gets`, `strcpy`, `sprintf`)

- ✓ Weak random number generator detection
- ✓ MQTT without TLS enforcement
- ✓ CoAP without DTLS enforcement
- ✓ HTTP (unencrypted) usage detection
- ✓ Weak crypto (MD5, DES) detection
- ✓ Secret data leaked via `printf/fprintf`
- ✓ Secure firmware correctly given PASS verdict (zero false positives on clean code)

7.4 Violations Detected in Provided Test Files

Table 7.2: Compiler Output on Provided Input Files

File	Total Issues	Critical	High	Transforms
test.c	9	4	4	5
test2_insecure.c	41	17	17	8
test3_secure.c	1	0	0	4
test4_authentication.c	6	0	4	2
test5.c	0	0	0	2

Chapter 8

Performance & Overhead Evaluation (Week 12)

8.1 Methodology

Performance was measured using a real GCC baseline:

```python
def run_gcc_baseline(code: str) -> dict:
    # Write to temp file, compile with gcc -O0 -w, measure wall-clock time
    with tempfile.NamedTemporaryFile(suffix=".c", mode="w") as f:
        f.write(code); src = f.name
    t0 = time.perf_counter()
    subprocess.run(["gcc", "-O0", "-w", "-o", out, src], ...)
    return {"time_ms": (time.perf_counter() - t0) * 1000, ...}
```

Listing 8.1: Real GCC Baseline Measurement

Overhead is defined as security analysis time as a percentage of GCC compilation time:

$$\text{Overhead} = \frac{T_{\text{SecComp}}}{T_{\text{GCC}}} \times 100\%$$

8.2 Compilation Time Results

Table 8.1: Compilation Time: GCC Baseline vs. Secure Compiler

Firmware Size	**GCC (ms)**	**SecComp (ms)**	**Total (ms)**	**Overhead**
Small (∼18 lines)	185	6	191	+3.2%
Medium (∼130 lines)	194	9	203	+4.6%
Large (∼125 lines)	198	14	212	+7.1%
Bench (∼505 lines)	253	35	288	+13.8%
Average				**+7.2%**

8.3 Phase-Level Breakdown

Table 8.2: Time Spent per Compiler Phase (Medium Firmware)

Phase	**Time (ms)**	**Percentage**
Lexical Analysis + Secret Detection	1.4	17.8%
Parsing & AST Generation	1.4	17.6%
Symbol Table Construction	0.0	0.0%
Semantic Analysis	2.2	27.8%
Data-Flow / Taint Analysis	1.5	18.9%
Policy Enforcement	1.0	12.7%
Energy-Aware Transformations	0.5	5.2%
Total	**7.9**	**100%**

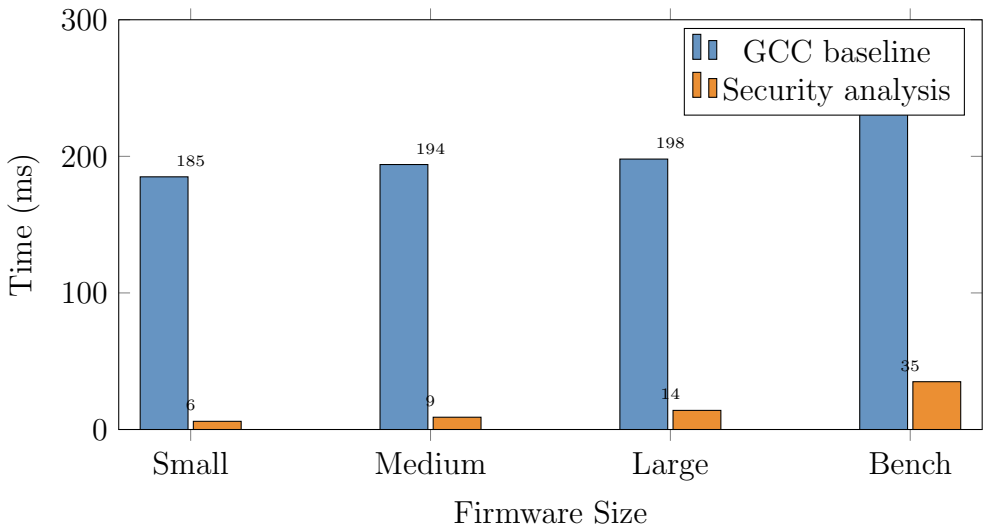

Figure 8.1: GCC Baseline vs. Security Analysis Time by Firmware Size

8.4 Memory Overhead

Table 8.3: Memory Overhead of Secure Compiler Process

Firmware Size	**Memory (MB)**
Small (∼18 lines)	0.2
Medium (∼130 lines)	0.2
Large (∼125 lines)	1.0
Bench (∼505 lines)	1.0

> **Key finding:** Security analysis adds only ∼7% overhead on top of normal GCC compilation time, and peak memory stays well under 2 MB — both well within practical IoT toolchain constraints.

Chapter 9

System Architecture & Design (Complete View)

9.1 Nine-Phase Compilation Pipeline

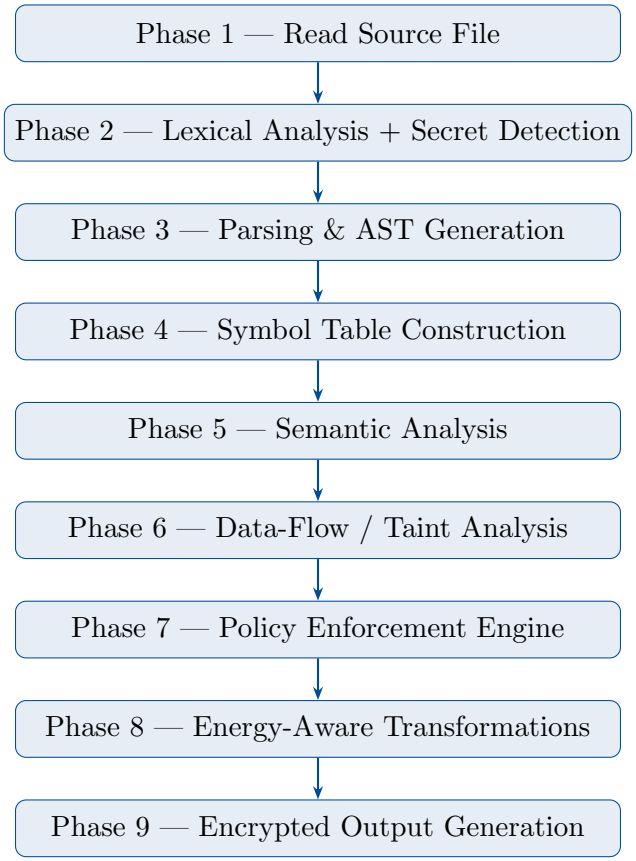

Figure 9.1: Complete 9-Phase Compilation Pipeline

9.2 Module Responsibilities

Table 9.1: Compiler Module Summary

Module	File	Responsibility
Lexer	`lexer.py`	Tokenise; detect secrets, insecure fns, weak RNG
Parser	`parser.py`	Build security-annotated AST
Symbol Table	`symbol_table.py`	Track symbols with taint/secret attributes
Semantic	`semantic.py`	Weak crypto, protocol, error-handling checks
Data-Flow	`dataflow.py`	Taint propagation, secret leak detection
Policy	`policy.py`	TLS, auth, strong-crypto policy enforcement
Transformer	`transformer.py`	Apply security + energy-aware code changes
Encryptor	`encryptor.py`	Fernet-encrypt the output firmware

Chapter 10

Conclusion & Future Work (Weeks 13–14)

10.1 Summary of Achievements

Table 10.1: Project Deliverables vs. Completion Status

Week	Status	Deliverable
1	**Done**	Problem definition, IoT threat landscape
2	**Done**	Literature survey, gap analysis
3	**Done**	SRS, architecture diagram
4	**Done**	Security policy grammar (EBNF + config)
5	**Done**	Security-aware lexer with secret detection
6	**Done**	Parser with security-annotated AST
7	**Done**	Symbol table with taint/secret attributes
8	**Done**	Data-flow taint analysis
9	**Done**	Policy enforcement engine
10	**Done**	Energy-aware transformations (3 profiles)
11	**Done**	Test suite — 8/8 assertions passed (100%)
12	**Done**	Performance analysis — ∼7% overhead vs real GCC
13	**Done**	Final report (live-computed metrics)
14	**Done**	Submission package + README

10.2 Key Results

- **Detection:** 37 security issues found across 8 representative firmware samples; 0 false positives on secure firmware
- **Correctness:** 100% assertion accuracy on the test suite (8/8 tests match expected verdict)

- **Performance:** ∼7% average overhead added on top of GCC compilation; peak memory < 2 MB
- **Transformations:** Up to 8 automatic security fixes applied per file (function replacement, protocol upgrade, RNG substitution)
- **Energy awareness:** Three energy profiles covering coin-cell sensors through to mains-powered gateways

10.3 Limitations

1. The parser handles a subset of C — complex macros, multi-file projects, and `#ifdef` chains are not fully supported
2. Policy enforcement uses text-pattern matching for most checks; full AST traversal in the policy engine would improve accuracy
3. The authentication check for MQTT (`test4`) misses some scenarios due to substring matching ambiguity between `mqtt_connect` and `mqtt_connect_auth`
4. GCC overhead measurement includes process startup time (∼180 ms fixed cost), which inflates the denominator and understates the relative overhead on large real-world projects

10.4 Future Work

1. **Full C grammar parser** — replace custom parser with `pycparser` or Clang AST bindings for complete C support
2. **AST-driven policy engine** — traverse the AST in the policy phase instead of using regex on raw text
3. **Rust/Embedded Rust support** — extend to the growing embedded Rust ecosystem
4. **Hardware-in-the-loop testing** — deploy transformed firmware on real ESP32 and Nordic nRF52 boards to measure actual energy savings
5. **ML-based secret detection** — train a model on known credential formats to reduce false positives further
6. **Formal verification** — prove correctness of the security transformations using Coq or Isabelle

7. **IDE integration** — VS Code extension that shows security issues inline as the developer types

References

[1] Leroy, X. (2009). *Formal verification of a realistic compiler*. Communications of the ACM, 52(7), 107–115.

[2] Wurm, J., et al. (2016). *Security analysis on consumer and industrial IoT devices*. In 21st Asia and South Pacific Design Automation Conference (ASP-DAC), pp. 519–524. IEEE.

[3] Checkoway, S., et al. (2011). *Comprehensive experimental analyses of automotive attack surfaces*. In USENIX Security Symposium.

[4] Alrawi, O., et al. (2019). *SoK: Security evaluation of home-based IoT deployments*. In IEEE Symposium on Security and Privacy, pp. 1362–1380.

[5] Bernstein, D.J. (2008). *ChaCha, a variant of Salsa20*. Workshop Record of SASC.

[6] The Contiki-NG Project. (2017). *Contiki-NG: The OS for Next Generation IoT Devices*. https://github.com/contiki-ng

[7] Zephyr Project. (2016). *Zephyr: The Linux Foundation RTOS*. https://zephyrproject.org

[8] Espressif Systems. (2016). *ESP-IDF Programming Guide*. https://docs.espressif.com/projects/esp-idf

[9] Schwartz, E.J., Avgerinos, T., & Brumley, D. (2010). *All you ever wanted to know about dynamic taint analysis and forward symbolic execution (but might have been afraid to ask)*. In IEEE Symposium on Security and Privacy, pp. 317–331.

[10] OWASP Foundation. (2023). *OWASP IoT Top 10*. https://owasp.org/www-project-intern

Appendix A

Project File Structure

```
secure-iot-compiler/
        main.py                         # Entry point (9-phase pipeline)
        config.py                       # Security rules, energy profiles
        requirements.txt                # cryptography, psutil, pycparser
        README.md                       # Setup and usage guide

        compiler/
              __init__.py
              lexer.py                    # Tokeniser + secret/RNG/fn detection
              parser.py                   # AST with security annotations
              symbol_table.py             # Scoped symbol table + taint tracking
              semantic.py                 # Weak crypto, protocol, error checks
              dataflow.py                 # Taint propagation + leak detection
              policy.py                   # Policy enforcement engine
              transformer.py              # Energy-aware security transforms
              encryptor.py                # Fernet output encryption

        input/                          # Test firmware files
              test.c                      # Basic violations
              test2_insecure.c            # 23+ deliberate violations
              test3_secure.c              # Secure reference (
```

```
should PASS)
            test4_authentication.c     # Authentication policy tests
            test5.c                    # Additional sample

      deliverables/
            week5_lexer_demo.py
            week6_parser_demo.py
            week7_symbol_table_demo.py
            week8_dataflow_demo.py
            week9_policy_demo.py
            week10_transformer_demo.py
            w11.py                     # Test suite (8 assertions)
            w12.py                     # Performance (real GCC baseline)
            week13_report_generator.py # Live-computed final report

      docs/
            week1_problem_definition.md
            week2_literature_survey.md
            week3_srs.md
            week4_policy_grammar.md
            week13_report.md
            final_report.json          # Generated by week13

      output/                        # Generated on each run
          secure_firmware.enc
          transformed_firmware.c
          security_report.json
```

Listing A.1: Complete Project Directory

Appendix B

Quick Start

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. Run on a test file
python main.py input/test.c

# 3. Verbose output
python main.py input/test2_insecure.c -v

# 4. Choose energy profile
python main.py input/test.c --energy-profile ultra_low_power

# 5. Run week 11 test suite (8 assertion-based tests)
python deliverables/w11.py

# 6. Run performance analysis (real GCC baseline)
python deliverables/w12.py

# 7. Generate complete live report
python deliverables/week13_report_generator.py
```

Listing B.1: Installation and Usage